

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

Authorization: Bearer <access_token>

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- **Per Client IP or API Key:**
 - **Standard Tier:** 1000 requests per hour.

- **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
 - 500 requests per minute.

2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.
 - `X-Rate`

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- **Per Client IP or API Key:**
 - **Standard Tier:** 1000 requests per hour.
 - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
 - 500 requests per minute.

2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.
 - `X-Rate`

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce ****HTTPS**** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- ****Per Client IP or API Key:****
 - ****Standard Tier:**** 1000 requests per hour.
 - ****Premium Tier:**** 10,000 requests per hour.
- ****Per User (if applicable):****
 - 500 requests per minute.

2.3. Implementation

- Use ****HTTP headers**** to communicate rate limit status:
 - ``X-RateLimit-Limit``: Total allowed requests in the current window.
 - ``X-RateLimit-Remaining``: Remaining requests.
 - ``X-Rate``

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- ****Use OAuth 2.0**** for secure, standardized, and scalable authentication.
- Support ****Bearer Tokens**** in the ``Authorization`` header:

```

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **\*\*API Keys\*\*** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **\*\*HTTPS\*\*** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **\*\*Per Client IP or API Key:\*\***
  - **\*\*Standard Tier:\*\*** 1000 requests per hour.
  - **\*\*Premium Tier:\*\*** 10,000 requests per hour.
- **\*\*Per User (if applicable):\*\***
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **\*\*HTTP headers\*\*** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:  
``

Authorization: Bearer <access\_token>  
``

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.

- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.

- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:  
---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.

- `X-Rate

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

### ## 1. Authentication Approach

#### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

### ## 2. Rate Limits

#### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

#### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**

- **Standard Tier:** 1000 requests per hour.
- **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **\*\*API Keys\*\*** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **\*\*HTTPS\*\*** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **\*\*Per Client IP or API Key:\*\***
  - **\*\*Standard Tier:\*\*** 1000 requests per hour.
  - **\*\*Premium Tier:\*\*** 10,000 requests per hour.
- **\*\*Per User (if applicable):\*\***
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **\*\*HTTP headers\*\*** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:  
``

Authorization: Bearer <access\_token>

`

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **\*\*HTTP headers\*\*** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

### ## 1. Authentication Approach

#### ### 1.1. Authentication Method

- **\*\*Use OAuth 2.0\*\*** for secure, standardized, and scalable authentication.
- Support **\*\*Bearer Tokens\*\*** in the `Authorization` header:  
...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **\*\*API Keys\*\*** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **\*\*HTTPS\*\*** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

### ## 2. Rate Limits

#### ### 2.1. Purpose

- Protect API resources from abuse.

- Ensure fair usage among clients.
- Maintain service availability and performance.

#### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

#### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

### # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.

- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.

- `X-RateLimit-Remaining`: Remaining requests.
- `X-Rate

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

### ## 1. Authentication Approach

#### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

### ## 2. Rate Limits

#### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

#### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

### ## 1. Authentication Approach

#### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.

- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.

- Support **Bearer Tokens** in the `Authorization` header:  
``

Authorization: Bearer <access\_token>  
``

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**

- 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:  
---

Authorization: Bearer <access\_token>  
---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

#### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

#### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

### # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.

- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - ``X-RateLimit-Limit``: Total allowed requests in the current window.
  - ``X-RateLimit-Remaining``: Remaining requests.
  - ``X-Rate``

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the ``Authorization`` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **\*\*API Keys\*\*** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **\*\*HTTPS\*\*** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **\*\*Per Client IP or API Key:\*\***
  - **\*\*Standard Tier:\*\*** 1000 requests per hour.
  - **\*\*Premium Tier:\*\*** 10,000 requests per hour.
- **\*\*Per User (if applicable):\*\***
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **\*\*HTTP headers\*\*** to communicate rate limit status:
  - ``X-RateLimit-Limit``: Total allowed requests in the current window.
  - ``X-RateLimit-Remaining``: Remaining requests.
  - ``X-Rate``

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:

- `X-RateLimit-Limit`: Total allowed requests in the current window.
- `X-RateLimit-Remaining`: Remaining requests.
- `X-Rate`

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

### ## 1. Authentication Approach

#### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

### ## 2. Rate Limits

#### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).

- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

Authorization: Bearer <access\_token>

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## 2. Rate Limits

### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.

- **\*\*Per User (if applicable):\*\***
- 500 requests per minute.

### ### 2.3. Implementation

- Use **\*\*HTTP headers\*\*** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **\*\*Use OAuth 2.0\*\*** for secure, standardized, and scalable authentication.
- Support **\*\*Bearer Tokens\*\*** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **\*\*API Keys\*\*** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **\*\*HTTPS\*\*** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

# API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **\*\*HTTPS\*\*** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **\*\*Per Client IP or API Key:\*\***
  - **\*\*Standard Tier:\*\*** 1000 requests per hour.
  - **\*\*Premium Tier:\*\*** 10,000 requests per hour.
- **\*\*Per User (if applicable):\*\***
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **\*\*HTTP headers\*\*** to communicate rate limit status:
  - ``X-RateLimit-Limit``: Total allowed requests in the current window.
  - ``X-RateLimit-Remaining``: Remaining requests.
  - ``X-Rate``

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **\*\*Use OAuth 2.0\*\*** for secure, standardized, and scalable authentication.
- Support **\*\*Bearer Tokens\*\*** in the ``Authorization`` header:

---

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

### ## 2. Rate Limits

#### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

#### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

#### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

### # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```

Authorization: Bearer <access_token>

```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

### ## 1. Authentication Approach

#### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

### ## 2. Rate Limits

#### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate

## # API Design Guidelines Version 2

### ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

### ## 1. Authentication Approach

#### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

---

Authorization: Bearer <access\_token>

---

- For internal services or simple use cases, **API Keys** may be used but should be limited to

#### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

#### ### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

### ## 2. Rate Limits

#### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

#### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.

- **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

```
Authorization: Bearer <access_token>
```

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### 1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **Per Client IP or API Key:**
  - **Standard Tier:** 1000 requests per hour.
  - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
  - `X-RateLimit-Limit`: Total allowed requests in the current window.
  - `X-RateLimit-Remaining`: Remaining requests.
  - `X-Rate`

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access\_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

### ### 1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

### ### 1.3. Authentication Security

- Enforce **\*\*HTTPS\*\*** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

---

## ## 2. Rate Limits

### ### 2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

### ### 2.2. Default Rate Limits

- **\*\*Per Client IP or API Key:\*\***
  - **\*\*Standard Tier:\*\*** 1000 requests per hour.
  - **\*\*Premium Tier:\*\*** 10,000 requests per hour.
- **\*\*Per User (if applicable):\*\***
  - 500 requests per minute.

### ### 2.3. Implementation

- Use **\*\*HTTP headers\*\*** to communicate rate limit status:
  - ``X-RateLimit-Limit``: Total allowed requests in the current window.
  - ``X-RateLimit-Remaining``: Remaining requests.
  - ``X-Rate``

## # API Design Guidelines Version 2

## ## Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

---

## ## 1. Authentication Approach

### ### 1.1. Authentication Method

- **\*\*Use OAuth 2.0\*\*** for secure, standardized, and scalable authentication.
- Support **\*\*Bearer Tokens\*\*** in the ``Authorization`` header:

```

Authorization: Bearer <access_token>

- For internal services or simple use cases, ****API Keys**** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce ****HTTPS**** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- ****Per Client IP or API Key:****
 - ****Standard Tier:**** 1000 requests per hour.
 - ****Premium Tier:**** 10,000 requests per hour.
- ****Per User (if applicable):****
 - 500 requests per minute.

2.3. Implementation

- Use ****HTTP headers**** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.
 - `X-Rate

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

...

Authorization: Bearer <access_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- **Per Client IP or API Key:**
 - **Standard Tier:** 1000 requests per hour.
 - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
 - 500 requests per minute.

2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.
 - `X-Rate

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:
``

Authorization: Bearer <access_token>
``

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.

- Maintain service availability and performance.

2.2. Default Rate Limits

- **Per Client IP or API Key:**
 - **Standard Tier:** 1000 requests per hour.
 - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
 - 500 requests per minute.

2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.
 - `X-Rate`

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:
...

Authorization: Bearer <access_token>

...

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.

- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- **Per Client IP or API Key:**
 - **Standard Tier:** 1000 requests per hour.
 - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
 - 500 requests per minute.

2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.
 - `X-Rate`

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

Authorization: Bearer <access_token>

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- **Per Client IP or API Key:**
 - **Standard Tier:** 1000 requests per hour.
 - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
 - 500 requests per minute.

2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.
 - `X-Rate`

API Design Guidelines Version 2

Introduction

This document outlines the best practices and standards for designing RESTful APIs to ensure co

1. Authentication Approach

1.1. Authentication Method

- **Use OAuth 2.0** for secure, standardized, and scalable authentication.
- Support **Bearer Tokens** in the `Authorization` header:

Authorization: Bearer <access_token>

- For internal services or simple use cases, **API Keys** may be used but should be limited to

1.2. Token Management

- Tokens must have a configurable expiration time (recommended: 1 hour).
- Implement refresh tokens to allow clients to renew access tokens without re-authenticating.
- Use HTTPS to encrypt token transmission.

1.3. Authentication Security

- Enforce **HTTPS** for all API endpoints.
- Validate tokens on each request.
- Implement scope-based access control to restrict actions based on user permissions.
- Log authentication attempts and monitor for suspicious activity.

2. Rate Limits

2.1. Purpose

- Protect API resources from abuse.
- Ensure fair usage among clients.
- Maintain service availability and performance.

2.2. Default Rate Limits

- **Per Client IP or API Key:**
 - **Standard Tier:** 1000 requests per hour.
 - **Premium Tier:** 10,000 requests per hour.
- **Per User (if applicable):**
 - 500 requests per minute.

2.3. Implementation

- Use **HTTP headers** to communicate rate limit status:
 - `X-RateLimit-Limit`: Total allowed requests in the current window.
 - `X-RateLimit-Remaining`: Remaining requests.